indexes in the search request. For example, omitting the index name(s) on the search request tells the engine to search across all indexes:

```
GET _search
{
  "query": {
    "match": {
      "actors": "Pacino"
    }
  }
}
```

We can also use `GET */_search` or `GET _all/_search`, which is equivalent to the previous query. All these forms search across all indexes in the cluster.

When we search across multiple indexes, we may want a document found in one index to take precedence over the same document found in another index. That is, we may want to boost certain indexes over others when performing a search across multiple indexes. For that, we can attach an `indices_boost` object at the same level as the `query` object. We can input multiple indexes with the appropriate boost rating set on this `indices_boost` object.

To demonstrate, we can create two new indexes (`index_top` and `index_new`) and index the movie *The Shawshank Redemption* in them (the code is available with the book's files). Now that we have the same movie across three indexes, let's create the query with a requirement to enhance the document score obtained from `movies_top` so *The Shawshank Redemption* is the topmost result.

Listing 8.32 Boosting the score of a document

```
GET movies*/_search
{
  "indices_boost": [
    { "movies": 0.1},
    { "movies_new": 0},
    { "movies_top": 2.0}
  ],
  "query": {
    "match": {
      "title": "Redemption"
    }
  }
}
```

Lowers indices_boost to 0.1

Lowers indices_boost to 0

Increases indices_boost to 2.0

The query doubles the score if the document is found in Query DSL's `movies_top` and decreases it to 10% (0.1) of its original value for documents fetched from the `movies` index. Finally, we set `indices_boost` to `0` for `movies_new` documents which means effectively no boost is applied. If the document's original score is 0.2876821 in `movies_top`, for example, the new score is 0.5753642 (2 * 0.2876821). The other documents' scores are calculated based on the setting in the `indices_boost` object.